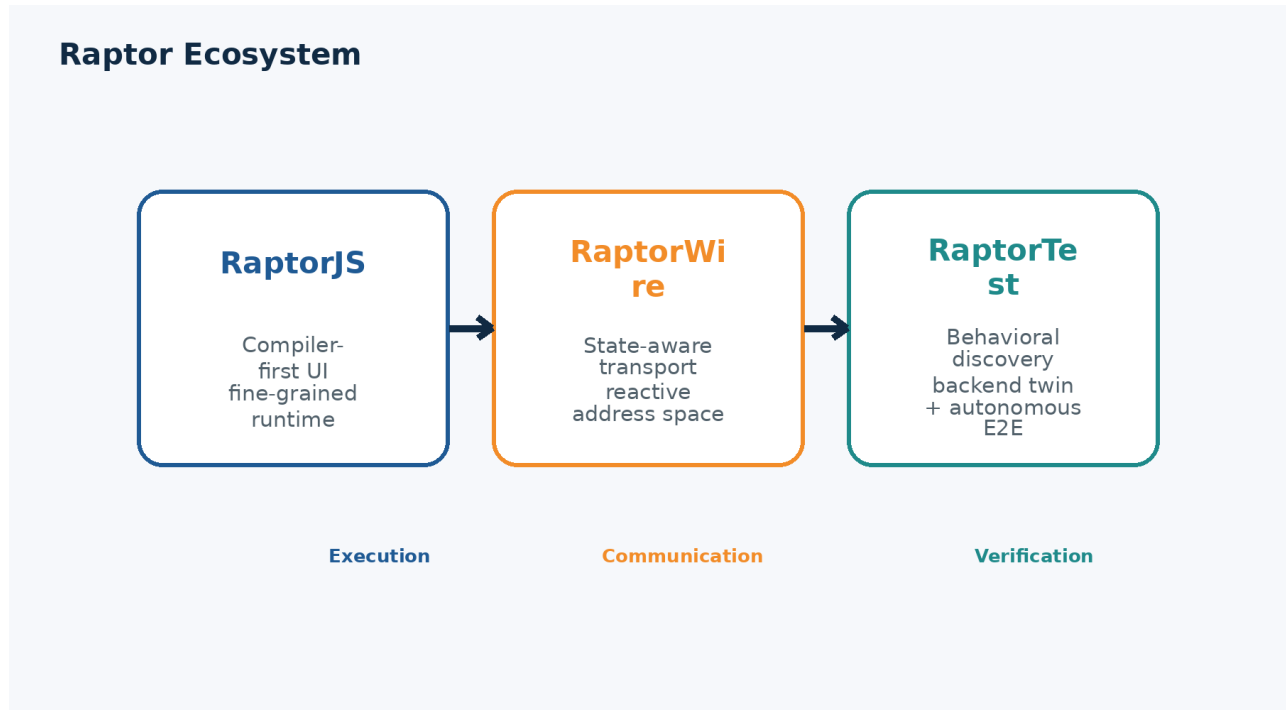


RAPTORTTEST

Autonomous Behavioral Testing & Backend Digital Twin

Technical Whitepaper v0.1



RaptorJS = execution, RaptorWire = communication, RaptorTest = verification.

Obiectiv: un framework de testare care descopera singur comportamentul unei aplicatii, sintetizeaza un backend fals stateful si exploreaza autonom spatiul de stari prin browserul real.

Concept / architecture draft - September 2026

Executive summary

RaptorTest este propus ca un sistem de testare E2E bazat pe descoperire comportamentala, nu pe colectii de scripturi imperative. In loc sa ceara dezvoltatorului sa descrie fiecare scenariu si fiecare mock, sistemul observa aplicatia, infereaza entitati, endpoint-uri, actiuni si tranzitii de stare, construiesc un backend digital twin stateful si foloseste browserul real pentru a explora sistematic aplicatia.

Ideea centrala este ca testul nu mai este un fisier care spune browserului unde sa dea click. Testul devine o executie derivata dintr-un model al aplicatiei. Acest model combina un Semantic UI Model, un Behavior Graph, un Contract Model si un Server State Model. Din acestea, RaptorTest poate genera scenarii happy-path, boundary, failure, race-condition si recovery, iar fiecare bug poate fi salvat ca o capsula de executie determinista si reprodus ulterior identic.

Pozitionarea intentionata nu este „AI care scrie Playwright”. Pozitionarea este: „motor care descopera modelul comportamental al aplicatiei, sintetizeaza backend-ul necesar si cauta autonom tranzitii neacoperite sau inconsistente”.

Pentru aplicatii RaptorJS/RaptorWire, sistemul poate folosi informatii white-box despre graful reactiv, server signals, Reactive Address Space si schemele de transport. Pentru aplicatii React, Vue, Angular, Svelte sau alte stack-uri, poate functiona in mod black-box / grey-box prin DOM semantic, DevTools Protocol, trafic de retea, surse TypeScript, OpenAPI/GraphQL si artefacte HAR.

Pilon	Rol principal	Rezultat
RaptorProbe	Descoperire si observare	Model initial al UI-ului, API-ului si starilor
Behavior Graph	Reprezentare executabila	Stari, actiuni, preconditii, efecte, tranzitii
RaptorTwin	Backend digital twin	API stateful, auth, DB virtuala, streams, timers
RaptorExplorer	Explorare autonoma	Scenarii alese dupa coverage, risc, noutate si cost
RaptorOracle	Verificare automata	Invariante, contracte, consistenta UI-server, drift
RaptorChaos	Fault injection	Latenta, timeout, errors, races, out-of-order, disconnect
RaptorReplay	Reproducere	Capsule deterministe, minimizare si replay

Document map

Capitol	Continut
1-3	Problema, principii si peisajul actual al testarii web
4-7	Arhitectura RaptorTest, Probe, Behavior Graph si Semantic UI Model
8-11	RaptorTwin, sinteza automata a mockurilor si modelarea backend-ului
12-15	Explorare autonoma, State Forking, temporal testing si chaos
16-19	Oracles, mutation testing, differential testing si coverage
20-23	Replay determinist, CI/CD, integrare RaptorJS/RaptorWire si securitate
24-28	CLI/DSL, format intern, algoritmi, benchmark-uri si roadmap
Anexe	Exemple de executie, structuri de date si referinte

1. Problema pe care o rezolva RaptorTest

Testarea end-to-end moderna are o contradicție structurală: este cea mai apropiată de comportamentul real al utilizatorului, dar devine rapid costisitoare, fragilă și greu de întreținut. Pentru a controla testele, echipele introduc seed-uri de baza de date, fixtures, mocks, interceptors, page objects, selectori stabili, setup de auth și utilitare pentru așteptarea request-urilor. Rezultatul este adesea o a doua aplicație - suita de testare - care trebuie menținută în paralel cu produsul.

Mock-urile statice reduc costul infrastructurii, dar pot devia de backend-ul real. Testele pe backend real cresc fidelitatea, dar cer medii, date, resetări și pot deveni lente sau nondeterministe. RaptorTest încearcă să elimine opoziția printr-un model intermediar: backend-ul fals este stateful, învățat și verificabil, nu o colecție de răspunsuri independente.

- Fragilitatea selectorilor și a scenariilor imperative.
- Duplicarea logicii de business în fixtures și mock handlers.
- Necesitatea de a reseta sau seed-ui baza de date pentru fiecare scenariu.
- Dificultatea de a reproduce race conditions, timeout-uri și ordini neobisnuite ale răspunsurilor.
- Coverage-ul traditional nu măsoară tranzițiile de stare sau comportamentul utilizatorului.
- Bug report-urile E2E contin adesea video/log-uri, dar nu un mediu executabil reproductibil.

Model traditional:

```
Developer -> scrie test -> scrie mock -> scrie fixture -> pregateste DB -> ruleaza browser  
-> assertion
```

RaptorTest:

```
Application -> observe -> infer -> synthesize Twin -> explore -> verify -> minimize ->  
replay
```

2. Principii de proiectare

1. Model-first, nu script-first. Scripturile pot exista, dar modelul comportamental este sursa principală pentru generarea execuțiilor.
2. Stateful by default. Un fake server trebuie să pastreze starea și să respecte efectele mutațiilor ulterioare.
3. Real browser, real client code. UI-ul trebuie să facă request-urile normale; aplicația nu ar trebui să știe că backend-ul este sintetic.
4. Determinism controlabil. Timpul, rețeaua, randomizarea și starea backend trebuie să poată fi înghețate și reluate.
5. Semantic identity peste selectori CSS fragili. Nodurile sunt identificate prin rol, context, acțiune și efect de rețea.
6. Coverage comportamental. Măsurăm stări, tranziții, erori și condiții de rețea, nu doar linii de cod.

- 7. Generare explicabila. Orice scenariu generat trebuie sa arate de ce a fost ales si ce ipoteza verifica.
- 8. Compatibility-first. Sistemul trebuie sa functioneze si fara RaptorJS, iar integrarea Raptor sa ofere doar inteligenta suplimentara.

Regula de produs: daca un feature nou nu reduce munca manuala, nu creste fidelitatea sau nu descopera o clasa noua de defecte, nu are prioritate.

3. Peisajul actual si spatiul de diferentiere

Instrumentele existente ofera componente valoroase, dar in mod normal lasa dezvoltatorul sa defineasca explicit testele, mock-urile sau schemele de intrare. Playwright poate intercepta/modifica request-uri si poate servi trafic din HAR. Cypress poate spiona si stub-ui request-uri, inclusiv raspunsuri intarziate sau erori. MSW ofera mocking la nivel de retea si are tooling pentru generarea de handlers din trafic HAR sau documente OpenAPI. Pact se concentreaza pe consumer-driven contract testing. Schemathesis genereaza teste API din OpenAPI/GraphQL si poate executa fluxuri stateful multi-step.

Capabilitate	Instrumente existente (exemple)	RaptorTest - tinta
Browser E2E	Playwright, Cypress	Da, dar scenariile pot fi generate din model
Request interception	Playwright, Cypress, MSW	Da + invatare si corelare cu stari UI
Replay trafic	HAR / MSW Source	Da + transforma replay-ul intr-un server stateful
Contract testing	Pact	Da + contracte inferate si verificate in executii UI
API generation/fuzzing	Schemathesis	Da + UI-driven state exploration
Backend fals stateful inferat	Partial / custom	Obiectiv central
Explorare autonoma a UI-ului	Tooling separat / AI agents	Obiectiv central, ghidat de Behavior Graph
State forking copy-on-write	Nu este paradigma standard E2E	Obiectiv central
Deterministic execution capsule	Artefacte disparate	Artefact unic de replay
Behavioral coverage	Partial	Metrici first-class

RaptorTest nu trebuie sa concureze prin duplicarea acestor capabilitati. Avantajul trebuie sa vina din compunerea lor intr-un ciclu autonom: observe -> infer -> synthesize -> explore -> verify -> replay.

4. Arhitectura de ansamblu

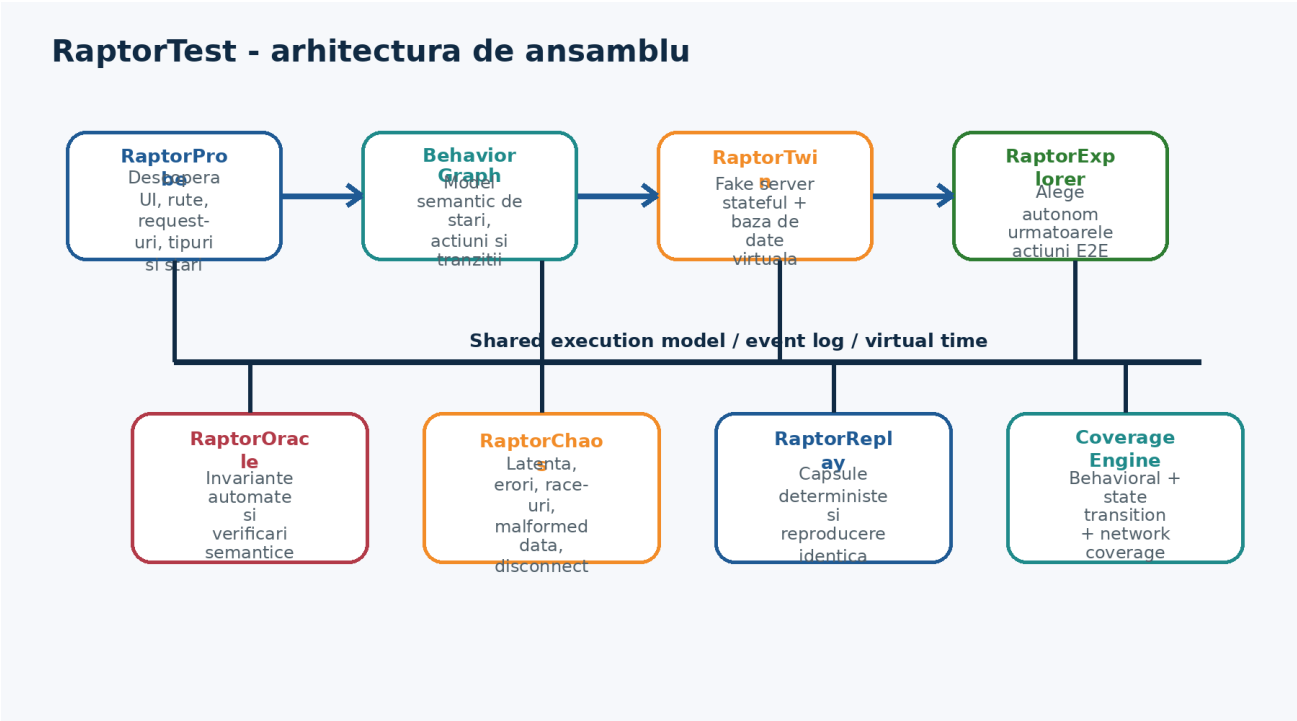


Figura 1. Componentele RaptorTest si bus-ul comun de executie.

Toate componentele comunica printr-un event log comun. Fiecare actiune de UI, request, response, schimbare de storage, timer, frame de WebSocket sau tranzitie de stare primeste un timestamp virtual si un identificator determinist. Acest jurnal este baza pentru inferenta, coverage, minimizarea bugurilor si replay.

Componenta	Intrare	Iesire
RaptorProbe	Browser + surse + trafic	Observatii normalizate
Behavior Graph	Observatii + contracte	Stari si tranzitii candidate
RaptorTwin	Model de server	API executabil + snapshots
RaptorExplorer	Graph + coverage	Actiuni/scenarii urmatoare
RaptorOracle	Executie + invariants	Pass/fail/warning + explicatie
RaptorChaos	Politici de fault	Evenimente de retea/timp mutate
RaptorReplay	Capsula	Executie determinista

5. Learning pipeline: observe -> infer -> synthesize

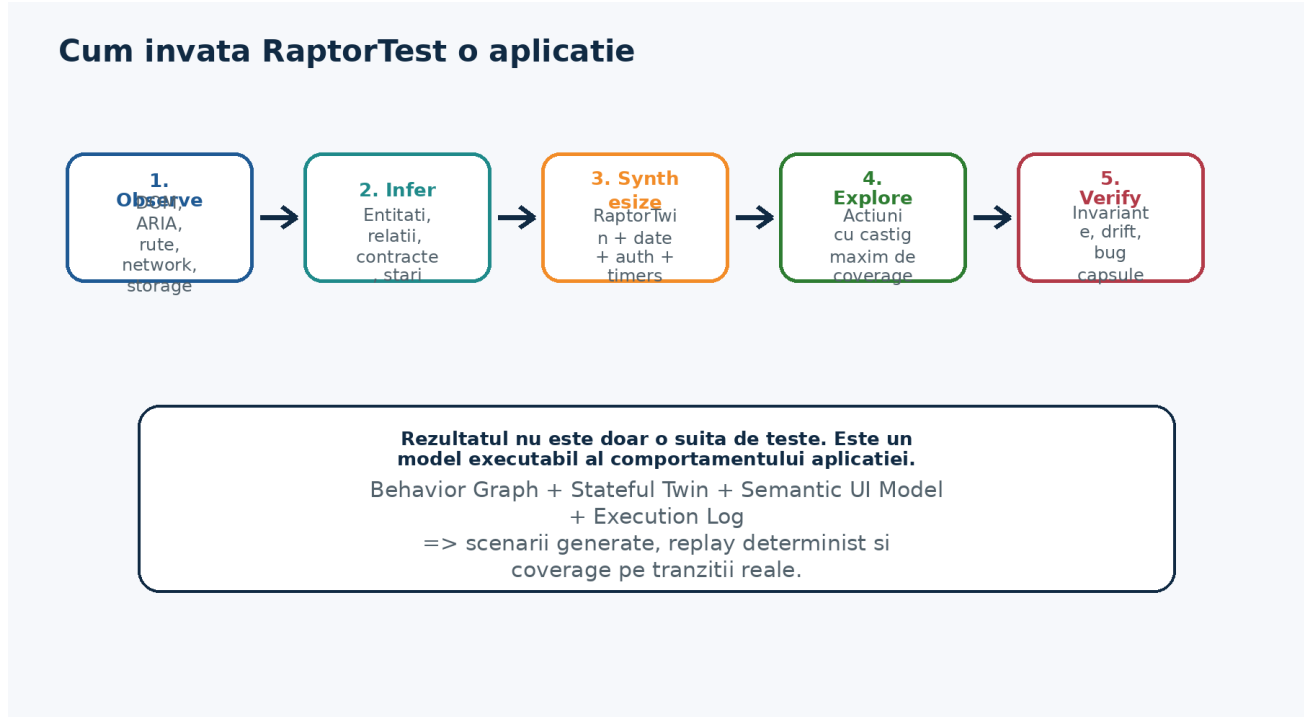


Figura 2. Pipeline-ul de invatare si executie.

Invatarea nu trebuie sa fie un proces AI opac. RaptorTest ar trebui sa combine reguli deterministe, inferenta statistica si optional un model LLM pentru semantica. Contractele, identificatorii si dependentele observabile se infera determinist; LLM-ul poate propune denumiri, scopul unui ecran sau clasificarea unei actiuni, dar aceste ipoteze raman marcate cu nivel de incredere.

Sursa	Ce poate furniza	Incredere tipica
DOM + ARIA	Roluri, labels, formulare, structura semantica	Ridicata
Network traffic	Endpoint, metoda, payload, response, timing	Foarte ridicata
TypeScript types	Shape-uri, enum-uri, optionalitate	Ridicata
OpenAPI / GraphQL	Contract explicit	Foarte ridicata
HAR	Exemple concrete de trafic	Ridicata pentru exemple
Source analysis	Call sites, route builders, query keys	Medie-ridicata
LLM semantic inference	Intent, categorii, denumiri	Variabila; trebuie verificata
RaptorJS metadata	Signals, bindings, queries, RAS	Foarte ridicata

6. RaptorProbe

RaptorProbe este stratul de observabilitate. Scopul sau nu este doar sa „crawl-uiasca” pagini, ci sa produca evenimente normalizate care pot fi corelate. Un click trebuie legat de request-urile provocate, de schimbarile vizibile in UI, de modificarile storage-ului si de tranzitia de stare rezultata.

- Instrumentare browser: DOM, accessibility tree, navigation, console, exceptions, performance events.
- Instrumentare network: HTTP/1.1, HTTP/2, HTTP/3 unde runner-ul permite, fetch/XHR, WebSocket, SSE, streaming.
- Observarea storage-ului: cookies, localStorage, sessionStorage, IndexedDB.
- Observarea timerelor si a microtask-urilor relevante pentru reproducere.
- Static analysis optional: import graph, route definitions, API clients, generated SDKs, TypeScript interfaces.
- Raptor-native probe: Reactive Address Space, serverSignal, query graph, DOM bindings si transaction boundaries.

```
probe.event = {
  id: "evt_9182",
  virtualTime: 1820,
  type: "network.response",
  correlation: ["ui_action_44", "request_87"],
  payload: { method: "POST", route: "/cart/items", status: 200 }
}
```

7. Behavior Graph

Behavior Graph este reprezentarea centrala a aplicatiei. Nodurile sunt stari observabile sau abstracte; muchiile sunt actiuni care pot schimba starea. Fiecare muchie are preconditii, efecte asteptate, cost de executie, risc, nivel de incredere si informatii de coverage.

```
State: Cart.Populated
  facts:
    auth = true
    cart.items >= 1

Action: Checkout
  source: button(role=button, name="Checkout")
  preconditions: cart.items > 0
  network: POST /checkout
  transitions:
    200 -> Checkout.Pending
    409 -> Cart.Conflict
    timeout -> Checkout.Unknown
```

Proprietate	Rol
State facts	Conditii care definesc starea curenta

Proprietate	Rol
Action identity	Identitate semantica, nu selector CSS brut
Preconditions	Cand actiunea este valida
Effects	Mutatii de server/UI asteptate
Transitions	Stari succesoare posibile
Confidence	Cat de sigur este modelul inferat
Coverage metadata	Vizitat/ne-vizitat, defecte, combinatii de fault
Cost	Durata estimata si resurse
Risk	Cat de importanta este tranzitia pentru produs

8. Semantic UI Model

RaptorTest trebuie sa evite dependenta primara de selectori CSS sau XPath. Fiecare element interactiv primeste o identitate semantica obtinuta din accessibility tree, text, context, componenta, request-uri asociate si efecte observate.

```
UI_NODE_918 = {
  role: "button",
  accessibleName: "Add to cart",
  context: ["ProductCard", "SKU:ABC-42"],
  actionEffect: "POST /cart/items",
  stability: 0.97
}
```

Self-healing-ul devine astfel o problema de re-identificare semantica. Daca textul se schimba din „Add to cart” in „Add”, dar rolul, contextul produsului si request-ul declansat raman aceleasi, sistemul poate propune aceeasi identitate cu un scor de incredere. Schimbarile ambigue nu sunt reparate tacit; sunt raportate.

Principiu: self-healing-ul nu trebuie sa ascunda regresii. Orice remapare semantica importanta este vizibila si auditabila.

9. RaptorTwin - backend digital twin

RaptorTwin - backend digital twin stateful

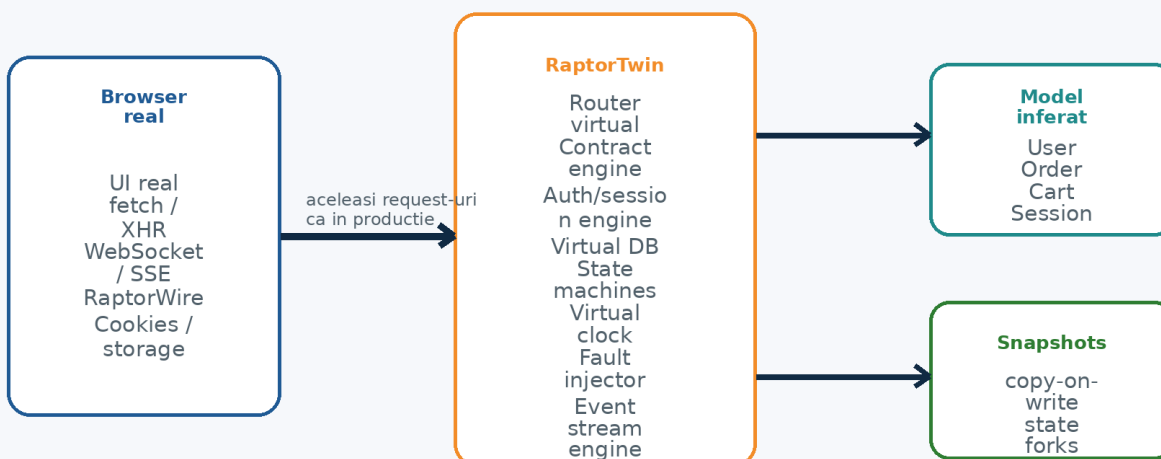


Figura 3. UI-ul ruleaza normal; request-urile sunt servite de un backend stateful sintetizat.

RaptorTwin trebuie sa fie suficient de expresiv pentru a modela efectele backend-ului, nu implementarea lui interna. Obiectivul nu este sa recreeze microserviciile reale, ci sa reproduca contractele si tranzitiile observabile relevante pentru client.

- Routing si matching pentru HTTP, GraphQL, WebSocket/SSE si RaptorWire.
- Virtual database cu entitati, relatii, constrangeri, indexuri logice si generatoare de ID.
- Auth/session engine cu token expiry, refresh, roles si revocation.
- Event/stream engine pentru notificari, subscriptions si server push.
- Virtual clock pentru expirari, debounce, retry, TTL si time travel.
- Fault injector pentru erori controlabile si reproductibile.
- Snapshot/fork engine cu copy-on-write.

10. Sinteza automata a mockurilor si a contractelor

RaptorTest poate porni de la patru situatii: backend real disponibil, numai trafic inregistrat, numai specificatie/schema sau doar frontend. In fiecare situatie construiesti un Contract Model cu niveluri diferite de incredere.

Input initial	Strategie de sinteza
Backend + browser	Reordeaza request/response, coreleaza secvente si infera efecte
HAR	Grupeaza rute, parametri si exemple; incearca sa identifice resurse stateful
OpenAPI/GraphQL	Genereaza tipuri/rute si foloseste exemple sau generatoare pentru date
TypeScript frontend	Extrage tipuri DTO, API clients si call sites; genereaza contracte candidate
RaptorJS app	Foloseste metadata compiler/query/RAS pentru contracte precise

Inferenta CRUD este doar inceputul. Motorul trebuie sa observe dependente. Daca `POST /orders` returneaza `id=91`, iar urmatoarea navigare foloseste `GET /orders/91`, apare o relatie producer-consumer. Daca `PATCH /profile` determina ulterior ca `GET /me` sa returneze date noi, Twin-ul invata efectul transversal.

Inference candidate:

```
POST /orders -> creates Order{id}
GET /orders/:id -> reads Order{id}
PATCH /orders/:id -> updates Order{id}
DELETE /orders/:id -> removes Order{id}
```

confidence = 0.94

evidence = 18 observed sessions

11. Data generation si adaptive fixtures

Fixture-urile nu ar trebui sa fie fisiere fixe, ci generatoare constranse de model. Pentru fiecare camp, sistemul poate pastra tipul, domeniul observat, exemple, boundary values si relatii cu alte entitati. Astfel poate genera atat date valide, cat si mutatii intentionat invalide.

Tip inferat	Generator normal	Mutatii de margine
string email	email sintetic valid	gol, foarte lung, unicode, invalid
uint32 id	ID existent / nou	0, max, duplicate, absent
enum status	valoare din enum	necunoscut, case variant
money decimal(2)	interval observat	0, negativ, foarte mare, rounding
datetime	aproape de now	expired, future, DST boundary
collection	0..N elemente	empty, max, duplicate

Datele generate pentru CI trebuie sa fie deterministe prin seed. Un failure trebuie sa poata fi reprodus cu acelasi seed chiar daca generatorul produce mii de combinatii.

12. RaptorExplorer - explorare autonoma

Explorarea nu trebuie sa fie random crawling. Fiecare actiune candidata primeste un scor care combina castigul de coverage, riscul functional, noutatea starii si costul de executie. Politicile pot fi ajustate pentru smoke, regression, chaos sau discovery.

```
score(action) =  
    coverageGain(action) * risk(action) * novelty(action)  
    -----  
    executionCost(action)
```

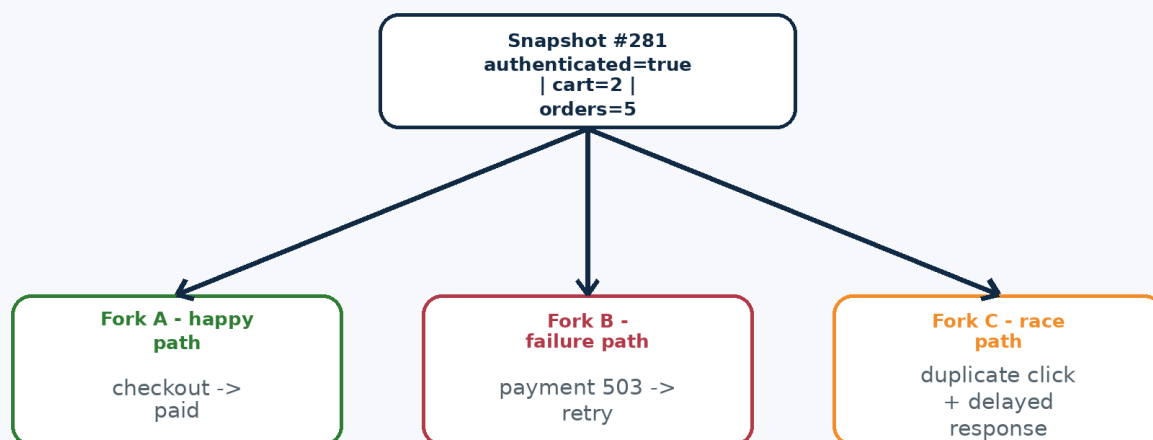
Semnal	Exemplu
Coverage gain	Actiunea conduce catre o tranzitie nevizitata
Risk	Checkout/payment are prioritate fata de schimbarea temei
Novelty	Combinatie de state facts nemaiobservata

Semnal	Exemplu
Execution cost	Flux de 2 pasi este preferat unuia de 20 daca acopera acelasi obiectiv
Instability	Traseele flaky pot primi rulari suplimentare pentru diagnostic

Pentru a evita explozia combinatorie, RaptorExplorer trebuie sa aplice partial-order reduction, state equivalence si risk-based pruning. Doua stari vizual diferite dar echivalente semantic pentru tranzitia testata pot fi colapsate.

13. State Forking si snapshots copy-on-write

State Forking - testare paralela fara reseed complet



Copy-on-write: fiecare fork modifica doar diferentele fata de snapshotul comun.

Figura 4. Un snapshot comun poate alimenta mai multe scenarii izolate.

State Forking elimina o mare parte din costul de setup. Dupa ce aplicatia ajunge intr-o stare scumpa - de exemplu utilizator autentificat, cos populat si date complexe incarcate - Twin-ul produce un snapshot. Mai multe scenarii pot porni din acelasi snapshot, fiecare primind o vedere copy-on-write.

- Reducerea timpului de autentificare si pregatire a datelor.
- Izolare completa intre scenarii fara resetarea intregului mediu.
- Paralelizare eficienta.
- Reproducere simpla: bug capsule poate include doar snapshot ID + delta.
- Posibilitatea de a explora mai multe rezultate ale aceleiasi actiuni dintr-o stare identica.

14. Temporal testing si virtual time

O clasa importanta de buguri depinde de timp: raspunsuri sosite in alta ordine, token expiry, retry, debounce, polling, optimistic updates sau animatii. RaptorTest trebuie sa controleze atat timpul serverului Twin, cat si timpul browserului, pe cat posibil.

```
scenario "out-of-order dashboard" {
  delay GET /orders = 1200ms
  delay GET /profile = 20ms
  delay GET /balance = 3100ms
  reorder responses = enabled
  virtualClock = deterministic
}
```

Control temporal	Utilitate
Freeze time	Teste de expirare si date dependente de clock
Advance time	TTL, retry/backoff, polling, debounce
Delay per route	Loading states si races
Response reordering	Stale overwrite / last-write-wins bugs
Network partition	Offline/reconnect
Frame loss	Protocoale reactive / stream recovery

15. RaptorChaos

RaptorChaos genereaza conditii adverse controlate, dar trebuie sa fie context-aware. A testa 500 pentru fiecare request este adesea inutil. Motorul foloseste contractul si Behavior Graph pentru a alege defectele cu impact probabil asupra tranzitiei curente.

Clasa	Exemple
HTTP	400, 401, 403, 404, 409, 422, 429, 500, 502, 503
Transport	timeout, reset, offline, partial body, malformed body
Timing	slow response, response order inversion, duplicate response
Streaming	lost frame, duplicate frame, out-of-order frame, reconnect gap
Data	missing field, null, unknown enum, duplicate entity, stale version
User interaction	double click, rapid navigation, submit during load

Pentru RaptorWire, RaptorChaos poate manipula frame ID, transaction boundaries si resync. De exemplu, daca frame 992 lipseste intre 991 si 993, sistemul verifica daca clientul detecteaza gap-ul si cere resynchronization conform protocolului.

16. RaptorOracle - verificari fara assertions manuale

Assertions manuale raman posibile, dar RaptorOracle trebuie sa ofere un set de invariante universale si derivate din model. Astfel, un scenariu generat poate descoperi defecte chiar daca nimeni nu a scris `expect(...)`.

- Fara uncaught exceptions sau unhandled promise rejections.
- Fara request loops neasteptate sau retry storms.
- Fara loading state imposibil de terminat dupa ce toate dependentele au raspuns.
- O mutatie confirmata de server trebuie sa fie reflectata in starea UI relevanta.
- O entitate stearsa nu trebuie sa ramana afisata ca activa dupa sincronizare.
- Navigarea trebuie sa conduca la un ecran semantic valid, nu la UI gol/broken.
- Actiunile interactive trebuie sa ramana accesibile prin roluri/labels, unde politica a11y o cere.
- Contractele request/response observate trebuie sa respecte modelul cunoscut.

Derived invariant example:

```
POST /cart/items -> 200
Twin state: cart.count 0 -> 1
UI binding: CartBadge.count
```

```
Oracle: eventually(UI.CartBadge.count == 1) within virtual deadline
```

17. Mutation testing la nivel de date si comportament

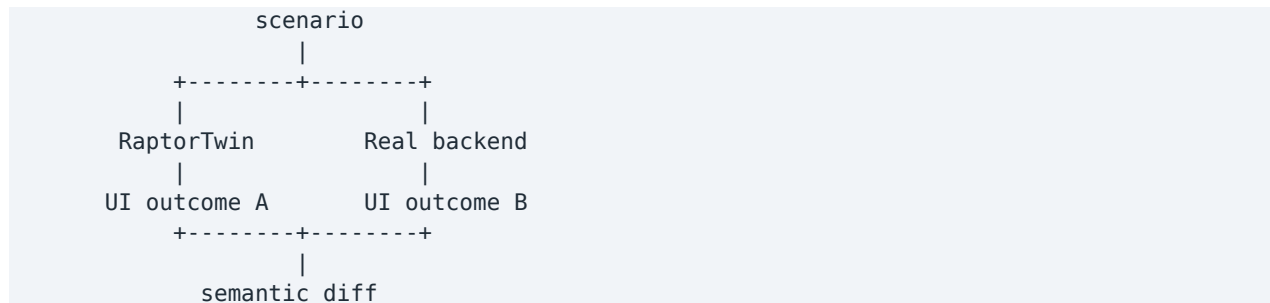
RaptorTest poate modifica intentionat contractele si efectele backend-ului pentru a verifica robustetea clientului. Spre deosebire de mutation testing clasic pe cod, aici mutatiile sunt aplicate observabilitatii serverului.

Mutatie	Intrebare testata
Camp lipsa / null	UI trateaza optionalitatea corect?
Enum necunoscut	Exista fallback?
ID duplicat	List reconciliation produce duplicate sau coruptie?
Version stale	Clientul detecteaza conflictul?

Mutatie	Intrebare testata
Succes fara body	UI depinde ilegal de un camp?
Efect intarziat	Optimistic update se reconciliaza?
Server push duplicat	Clientul aplica de doua ori mutatia?

18. Differential Testing: Twin vs backend real

Orice fake server risca sa devieze de productie. RaptorTest trateaza aceasta problema ca feature principal: acelasi flux poate rula impotriva RaptorTwin si a unui mediu real/staging, iar rezultatele sunt comparate semantic.



Dimensiune comparata	Exemple
Contract	status, fields, types, optionality
State transition	resource created/updated/deleted
UI outcome	route, visible state, semantic nodes
Timing class	immediate/eventual/streamed
Error semantics	retryable, auth, validation, conflict

Drift-ul este raportat ca diferenta intre modele, nu doar ca test esuat. Echipa poate decide daca Twin-ul se actualizeaza, backend-ul este regresat sau clientul depinde de un comportament nedocumentat.

19. Behavioral Coverage

Coverage-ul RaptorTest este multidimensional. Un procent unic poate exista pentru dashboard, dar diagnosticul trebuie sa arate ce tip de comportament ramane neexplorat.

Metric	Definitie propusa
UI State Coverage	stari semantice vizitate / stari cunoscute
Action Coverage	actiuni valide executate / actiuni cunoscute
Transition Coverage	tranzitii observate / tranzitii modelate
API Interaction Coverage	operatii API exercitate prin UI
Server State Coverage	categorii de stari backend atinse
Error State Coverage	tranzitii de eroare exercitate
Network Condition Coverage	fault/timing classes incercate
Invariant Coverage	invariante evaluate in contexte relevante

Behavioral coverage poate fi folosit de Explorer pentru a decide urmatorul scenariu. Aceasta creeaza un feedback loop: fiecare executie modifica harta de coverage si prioritatea actiunilor viitoare.

20. Deterministic Execution Capsule si RaptorReplay

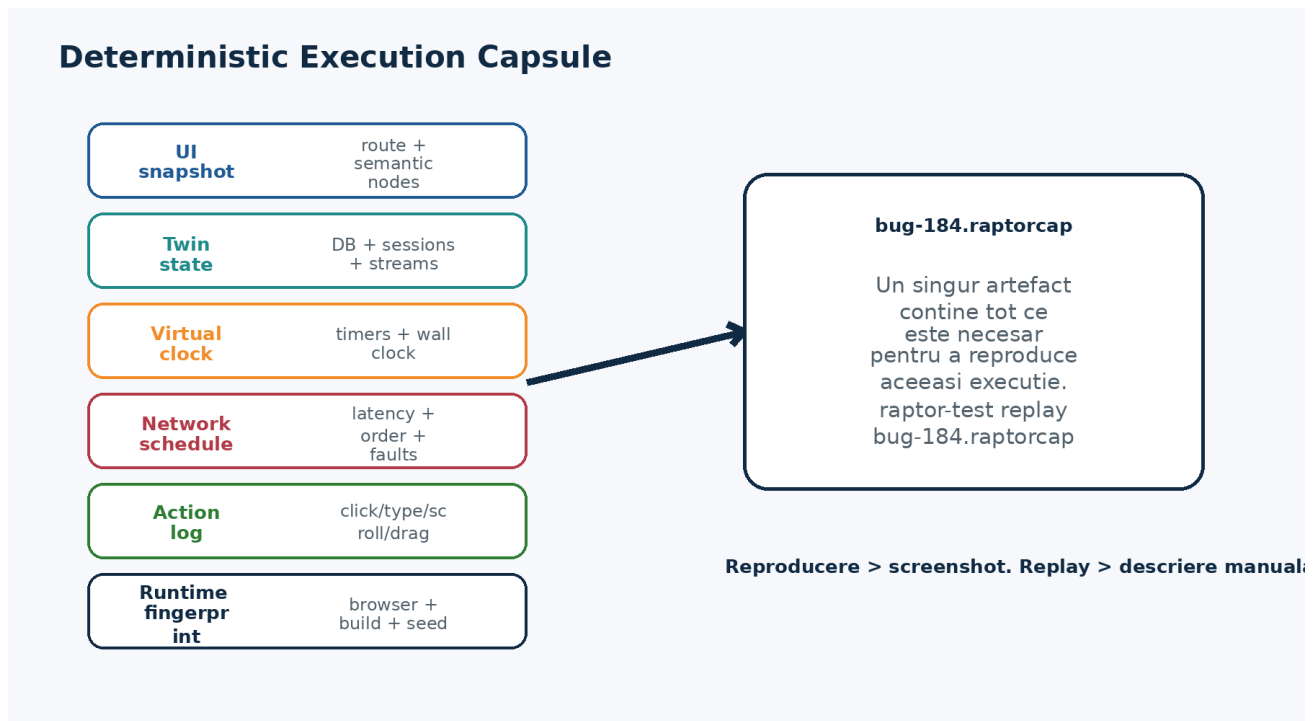


Figura 5. Un bug devine un artefact executabil, nu doar o descriere.

Capsula este unitatea de reproducere. Ea trebuie sa contina suficienta informatie pentru a reface aceeași stare si aceeași ordine de evenimente fara dependenta de serviciile originale.

Continut capsula	Exemplu
Build fingerprint	commit, bundle hash, browser version
Twin snapshot	entitati, sessions, stream cursors
Virtual time	epoch + timer queue
Network schedule	latency, order, injected faults
Action log	semantic node IDs + input values
Random seeds	data generator, exploration policy
Assertions/oracles	regulile care au esuat
Minimal trace	subsecventa minimizata care reproduce defectul

```
raptor-test replay bug-184.raptorcap
# optional
raptor-test replay bug-184.raptorcap --headed --pause-on-failure
```

21. Failure minimization

Un explorer autonom poate ajunge la bug dupa zeci de pasi. Pentru un developer, traseul este inutil de lung. RaptorTest trebuie sa aplice delta debugging asupra actiunilor, datelor si fault-urilor pentru a gasi o reproducere minima.

1. Elimina actiuni care nu modifica posibilitatea de a reproduce failure-ul.
2. Reduce payload-urile la campurile necesare.
3. Elimina fault-uri redundante si scurteaza delay-urile.
4. Gaseste cel mai apropiat snapshot precedent care pastreaza bug-ul.
5. Produce un scenariu minimal si o capsula compacta.

Tinta: fiecare defect automat trebuie sa vina cu „why it failed” + „smallest known reproduction”, nu doar cu un stack trace.

22. CI/CD si moduri de executie

Mod	Scop	Politica
Smoke	PR rapid	happy paths + high-risk transitions
Regression	PR / merge	coverage incremental + bug capsules cunoscute

Mod	Scop	Politica
Discovery	nightly	explorare pentru stari/tranzitii noi
Chaos	nightly / pre-release	fault injection + races + recovery
Differential	staging	Twin vs backend real
Replay	orice PR	toate capsulele deschise / fixed verification

CI trebuie sa foloseasca cache pentru modele si snapshots, dar fiecare artefact trebuie versionat cu schema si build-ul aplicatiei. Daca modelul este incompatibil cu noul build, RaptorProbe declanseaza re-learning incremental, nu folosirea tacita a unei harti vechi.

23. Integrarea cu RaptorJS si RaptorWire

RaptorTest trebuie sa fie framework-agnostic, dar ecosistemul Raptor ofera o cale white-box deosebit de puternica. Compilerul RaptorJS poate emite metadata de test care nu intra in bundle-ul production sau poate fi activata doar in build-uri de test.

- Mapping component/binding -> Semantic UI node.
- Mapping RAS address -> client signal -> DOM binding.
- Query graph si dependencies pentru server signals.
- RaptorWire schemas, transaction IDs si resync semantics.
- Derived signals si expected convergence points.

Raptor metadata (test build):

```
RAS 0x18A1 -> serverSignal("BTC.price") -> client slot 12 -> DOM binding 4
```

RaptorTest poate injecta:

```
SET 0x18A1 = 67231.45
```

si verifica direct daca binding 4 converge fara rerender neasteptat.

Aceasta combinatie permite „white-box intelligence + black-box execution”: motorul stie cum este construita aplicatia, dar actiunile si verificarile principale trec prin browser, asa cum ar face utilizatorul.

24. Securitate, confidentialitate si limite

Un instrument care inregistreaza trafic si genereaza snapshot-uri poate captura token-uri, date personale sau secrete. Privacy-by-design trebuie sa fie parte din formatul de artefact, nu un plugin optional.

- Redaction automata pentru Authorization, cookies sensibile, parole si campuri marcate secret.
- Allowlist/denylist pentru endpoint-uri si domenii.
- Data classification pentru fields personale si posibilitatea de sintetizare in loc de persistare.
- Criptarea capsulelor si snapshot-urilor la rest in CI.
- Interdictie implicita de a rula actiuni destructive pe production origins.
- Audit log pentru differential learning si update-uri ale Twin-ului.

Sistemul nu trebuie sa incerce sa ghiceasca tranzactii financiare sau actiuni destructive in medii reale fara reguli explicite. Discovery mode ar trebui sa aiba un safety policy care blocheaza submit-uri cunoscute ca ireversibile sau cere aprobarea dezvoltatorului.

25. CLI si developer experience

```
npx raptor-test init

npx raptor-test learn http://localhost:5173

npx raptor-test run

npx raptor-test run --mode chaos

npx raptor-test diff --twin --real https://staging.example.com

npx raptor-test replay bug-184.raptorcap
```

Comanda	Rol
init	configureaza proiectul si detecteaza framework-ul
learn	construieste/actualizeaza modelul comportamental
run	ruleaza politica implicita de testare
explore	ruleaza discovery concentrat pe coverage nou
chaos	injecteaza faults conform policy
diff	compara Twin cu backend real
replay	reproduce o capsula
inspect	deschide modelul, graph-ul si snapshots

26. Configuratie si DSL optional

Zero-config este o tinta, nu o dogma. Business invariants si zonele periculoase trebuie sa poata fi declarate compact.

```
export default defineRaptorTest({
  app: "http://localhost:5173",
  safety: {
    denyOrigins: ["https://production.example.com"],
    redact: ["authorization", "password", "cardNumber"]
  },
  priorities: {
    routes: ["/checkout", "/account"],
    transitions: ["Checkout.Pending -> Checkout.Paid"]
  },
  invariants: [
    invariant("paid-order-visible", ({ twin, ui }) =>
      twin.order.status === "paid" ? ui.has("OrderPaid") : true
    )
  ]
})
```

DSL-ul trebuie sa completeze modelul inferat, nu sa-l inlocuiasca. Cand developerul adauga o regula, aceasta devine o constrangere first-class in Behavior Graph si poate ghida Explorer-ul.

27. Model intern si formate

Artefact	Format propus	Observatie
Behavior Graph	.raptorgraph (JSON/CBOR in v0.x)	human-inspectable + compact option
Twin Model	.raptortwin	scheme, generators, state machines
Snapshot	.raptorsnap	content-addressed, copy-on-write
Execution Capsule	.raptorcap	archive determinist
Coverage	.rptorcov	multidimensional counters + graph metadata
Event Log	.raptorlog	append-only, timestamp virtual

```
interface Transition {
  id: string
  fromState: StateRef
  action: SemanticAction
  preconditions: Predicate[]
  networkEffects: NetworkEffect[]
  serverEffects: StateMutation[]
  expectedUI: UIInvariant[]
  confidence: number
  risk: number
  coverage: CoverageState
}
```

28. Algoritmi cheie

RaptorTest necesita algoritmi de graf si model checking mai mult decat prompt engineering. LLM-ul poate fi util pentru semantica, dar nucleul trebuie sa ramana reproductibil.

Problema	Abordare initiala
State equivalence	hash pe facts semantice + UI landmarks + Twin state class
Transition discovery	event correlation + causal windows + repeated evidence
Action selection	weighted best-first / UCB-like exploration
Scenario generation	path search constrans de preconditii
Failure minimization	delta debugging + snapshot bisection
Model drift	schema diff + behavioral diff
Race exploration	partial-order exploration pe request/timer events
Data generation	constraint-based + property-based generators

RaptorTest trebuie sa poata explica: „am ales aceasta actiune pentru ca acopera tranzitia T87, risc 0.91, neacoperita pana acum, cost estimat 1.2s”.

29. Benchmark-uri si criterii de succes

Nu este suficient sa demonstrem ca motorul poate produce teste. Trebuie masurate calitatea modelului, bug discovery, viteza si costul de mentenanta comparativ cu suite E2E scrise manual.

Metric	Tinta de cercetare / masurare
Twin fidelity	Procent de scenarii in care Twin si backend real au acelasi outcome semantic
Model precision	Tranzitii inferate corect / tranzitii inferate
Model recall	Tranzitii reale descoperite / tranzitii reale cunoscute
Bug yield	Defecte unice gasite per ora de explorare
Replay rate	Procent de failures reproductibile identic din capsula
Setup reduction	Timp de test setup evitat prin snapshots/forks
Maintenance delta	Modificari manuale in teste per schimbare de UI/API
Coverage gain	Behavioral transition coverage vs suita existenta
False-positive rate	Failures fara defect real

Comparațiile trebuie facute pe aceeași aplicație și aceleași resurse, cu baseline-uri precum Playwright/Cypress manual + mocks, backend real seed-uit și API property-based testing. Nu trebuie publicate afirmații de performanță până nu există benchmark-uri reproductibile.

30. Roadmap propus

Faza	Scope	Exit criteria
v0.1 - Probe	Browser events + HTTP + semantic nodes	Poate produce event log si harta de ecrane
v0.2 - Twin HTTP	Stateful REST fake DB + snapshots	CRUD si auth simplu pot rula fara backend
v0.3 - Behavior Graph	State/action/transition inference	Poate genera scenarii multi-step
v0.4 - Explorer	Coverage-guided E2E	Gaseste automat tranzitii nevizitate
v0.5 - Replay	Virtual time + deterministic capsule	Failure poate fi reprodus identic
v0.6 - Chaos	Fault/timing injection	Race/error matrix controlabila
v0.7 - Differential	Twin vs staging	Drift semantic raportat
v0.8 - Streams	WebSocket/SSE/RaptorWire	Stateful stream tests
v0.9 - Raptor native	RAS/query/binding introspection	White-box intelligence integrata
v1.0	Stability + plugins + CI scale	API public stabil si benchmark suite public

31. MVP: ce construim prima data

Pentru a evita construirea unui sistem prea mare inainte sa validam ideea, primul MVP trebuie sa demonstreze un lucru specific: un frontend CRUD poate fi invatat, backend-ul lui poate fi inlocuit de un Twin stateful, iar motorul poate genera singur cateva fluxuri E2E valide si un failure replayabil.

1. Runner Playwright ca executie browser initiala (nu reinventam browser automation in v0.1).
2. RaptorProbe pentru DOM/ARIA + fetch/XHR + navigation + console.
3. Contract inference pentru REST JSON simplu.
4. Virtual DB cu create/read/update/delete si relatii ID de baza.
5. Semantic UI identity pentru button/input/link/form.
6. Behavior Graph cu states/actions/transitions.
7. Explorer breadth-first + coverage bonus.
8. Oracle: JS errors, request errors, mutation->UI convergence.
9. Snapshot + deterministic seed.
10. Capsula simpla cu action log + Twin snapshot + network schedule.

MVP-ul nu are nevoie de AI pentru a demonstra valoarea. Daca nucleul determinist functioneaza pe o aplicatie reala, AI-ul poate fi adaugat ulterior pentru intent semantic, generare de date si prioritizare.

32. Structura initiala a repository-ului

```
raptor-test/
  packages/
    probe/           # browser instrumentation
    semantic-ui/     # stable semantic identities
    behavior-graph/  # state/action/transition model
    twin-core/       # router, virtual DB, auth, snapshots
    explorer/        # policy + search
    oracle/          # invariants
    chaos/           # faults and virtual network
    replay/          # capsule + deterministic execution
    coverage/        # behavioral metrics
    cli/             # developer interface
  adapters/
    playwright/
    raptorjs/
    openapi/
  examples/
  benchmarks/
  docs/
```

Separarea pe pachete permite folosirea componentelor independent. De exemplu, RaptorTwin poate fi folosit ca dev server fara Explorer, iar RaptorReplay poate deveni util chiar si pentru teste scrise manual.

33. Cazuri de utilizare prioritare

Aplicatie	De ce este potrivita
Admin / CRUD mare	Multe entitati si fluxuri repetitive, Twin usor de inferat
E-commerce	Cart/checkout are state transitions si multe edge cases
Trading / realtime dashboard	Timing, streams, out-of-order, stale state
Collaboration/chat	WebSocket, duplicate events, reconnect, ordering
IoT / monitoring	Streams dense, state convergence si fault injection
SaaS multi-role	Auth/permissions si state forks pe roluri
RaptorJS apps	Acces direct la reactive graph si RaptorWire

34. Riscuri tehnice si contramasuri

Risc	Impact	Contramasura
Inferenta gresita a backend-ului	False failures / Twin nerealist	confidence + differential validation
Explozie de state space	Cost enorm	equivalence, pruning, risk-based search
Self-healing mascheaza regresii	Bug ascuns	audit + confidence threshold + approval
Nondeterminism browser	Replay imperfect	virtual clock + network schedule + retries diagnostice
Date sensibile in traces	Security incident	redaction + encryption + policies
UI foarte canvas/WebGL	Semantic model slab	plugins + visual/coordinate adapter
Backend side effects externe	Twin incomplet	explicit external effect contracts
AI hallucination	Model fals	LLM optional, evidence-backed inference

35. Viziune: de la test runner la executable digital twin

Daca RaptorTest reuseste, produsul final nu va fi perceput doar ca un framework de testare. Behavior Graph + RaptorTwin formeaza un digital twin executabil al suprafetei client-server. Acest twin poate fi folosit pentru dezvoltare frontend inaintea backend-ului, demo-uri, reproducerea incidentelor, contract negotiation, migration testing si simularea unor volume sau conditii greu de creat pe sisteme reale.

In ecosistemul Raptor, cele trei produse formeaza un ciclu complet: RaptorJS compileaza si executa UI-ul, RaptorWire sincronizeaza starea, iar RaptorTest verifica automat comportamentul intregului sistem. Aceasta integrare poate deveni un avantaj structural, dar fiecare produs trebuie sa ramana suficient de modular pentru adoptie independenta.

Raptor Ecosystem

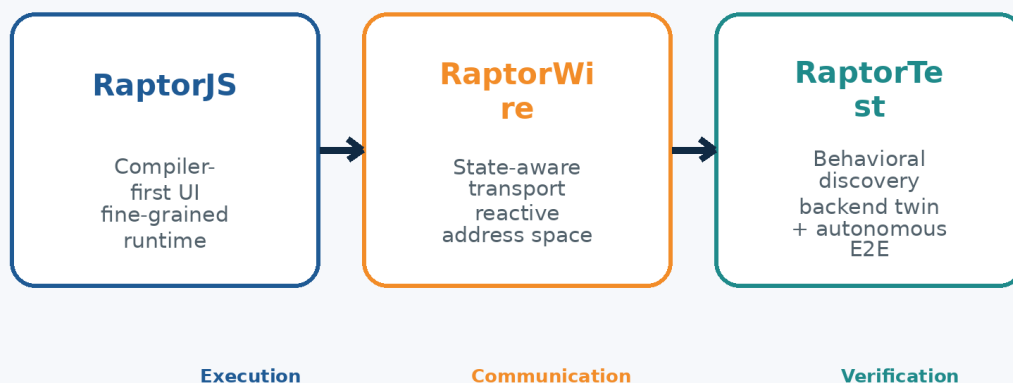


Figura 6. Viziunea ecosistemului Raptor.

North Star: „Aplicatia isi descrie comportamentul prin propria executie, iar sistemul de testare transforma aceasta executie intr-un model care poate fi explorat, falsificat, verificat si reprodus.”

Anexa A. Exemplu de bug report

RaptorTest

BUG RT-184

Scenario:

Product -> Add to cart -> immediate navigation -> Cart

Expected transition:

cart.count: 0 -> 1

Observed:

POST /cart/items 200 OK

Twin state: cart.count = 1

UI state after virtual 5s: cart.count = 0

Probable cause:

client state overwritten by stale GET /cart response

Minimal reproduction:

1. Open /products/81
2. Click Add to cart
3. Delay POST response 780 ms
4. Start GET /cart before POST commits
5. Let GET return after POST

Replay:

raptor-test replay RT-184.raptorcap

Anexa B. Exemplu de model Twin

```
entity User {
  id: uint32 @primary
  email: string @unique
  role: enum(admin,user)
}

entity Cart {
  id: uint32 @primary
  userId: ref(User.id)
  items: CartItem[]
}

route POST /cart/items {
  pre: session.userId != null
  effect: Cart(session.userId).items.append(body)
  response: { ok: true, count: cart.items.length }
}

route GET /cart {
  response: Cart(session.userId)
}
```

Anexa C. Exemplu de coverage report

Categorie	Covered	Known	Coverage
UI states	183	198	92.4%
User actions	428	491	87.2%
State transitions	972	1154	84.2%
API interactions	64	67	95.5%
Error states	71	116	61.2%
Network conditions	24	50	48.0%
Invariants contexts	318	366	86.9%

Valorile din exemplu sunt ilustrative, nu rezultate de benchmark ale unei implementari existente.

Anexa D. Referinte si tehnologii comparate

Sursa	URL	Relevanta
Playwright - Mock APIs	https://playwright.dev/docs/mock	Interceptare/modificare trafic HTTP/HTTPS si mocking din HAR.
Cypress - Intercepting network requests	https://docs.cypress.io/app/guides/network-requests	Spying/stubbing de request-uri, delay si control al raspunsurilor.
Mock Service Worker / MSW Source	https://source.mswjs.io/	Generare de request handlers din HAR/OpenAPI si replay de trafic.
Pact Docs	https://docs.pact.io/	Consumer-driven contract testing.
Schemathesis	https://schemathesis.io/	Generare property-based si stateful API testing din OpenAPI/GraphQL.

Aceste referinte sunt incluse pentru delimitarea zonei de produs. RaptorTest nu revendica drept noutate interceptarea de retea, mocking-ul, contract testing-ul sau generarea API de sine statatoare.

Diferentierea propusa este integrarea lor intr-un model comportamental executabil, cu backend twin stateful, explorare autonoma E2E, state forking si replay determinist.

Anexa E. Decizii de arhitectura deschise

- Formatul binar intern pentru event log si snapshots dupa v0.x.
- Daca virtual clock este implementat prin browser instrumentation, runtime shim sau ambele.
- Modelul exact de state equivalence pentru aplicatii foarte dinamice.
- Nivelul in care LLM-ul este permis sa propuna actiuni destructive in discovery.
- Strategia de plugin-uri pentru canvas, WebGL, Electron si mobile webviews.
- Interop cu Playwright traces si HAR pentru adoptie incrementală.
- Politica de versionare a RaptorTwin fata de backend contracts.
- Integrarea cu RaptorWire transactions si RAS in build-uri non-debug.